



**HỌC VIỆN
NGÂN HÀNG**
BANKING ACADEMY OF VIETNAM

CHƯƠNG 4 NGÔN NGỮ SQL

KHOA CÔNG NGHỆ THÔNG TIN VÀ KINH TẾ SỐ



1. Giới thiệu ngôn ngữ SQL
2. Truy vấn dữ liệu từ bảng đơn
3. Sử dụng hàm nhóm và mệnh đề having
4. Truy vấn dữ liệu từ nhiều bảng
5. Truy vấn lồng
6. Hợp dữ liệu từ nhiều câu truy vấn
7. Ngôn ngữ định nghĩa dữ liệu
8. Ngôn ngữ thao tác dữ liệu
9. Làm việc với view
10. Làm việc với Index



- SQL là một chuẩn của ANSI
(American National Standards Institute - Viện tiêu chuẩn quốc gia Hoa kỳ) về truy xuất các hệ thống CSDL. Các câu lệnh SQL được sử dụng để truy xuất và cập nhật dữ liệu trong một CSDL.
- SQL hoạt động với hầu hết các chương trình CSDL như MS Access, DB2, Informix, MS SQL Server, Oracle, Sybase v.v...



- **SQL dùng để quản lý dữ liệu trong các hệ quản trị CSDL quan hệ:**
 - SQL là viết tắt của **Structured Query Language** - Ngôn ngữ truy vấn cấu trúc.
 - SQL cho phép bạn truy cập vào CSDL.
 - SQL có thể thực thi các câu truy vấn trên CSDL.
 - SQL có thể lấy dữ liệu từ CSDL.
 - SQL có thể chèn dữ liệu mới vào CSDL.
 - SQL có thể xóa dữ liệu trong CSDL.
 - SQL có thể sửa đổi dữ liệu hiện có trong CSDL.



■ Lịch sử ngôn ngữ SQL

- **1970** -- Dr. Edgar F. "Ted" Codd của IBM được biết đến như là cha đẻ của Relational Database (Cơ sở dữ liệu quan hệ). Ông miêu tả một mô hình quan hệ (Relational Model) cho các Database.
- **1974** -- SQL (Structured Query Language) xuất hiện.
- **1978** -- IBM tiếp tục phát triển ý tưởng của Codd và công bố một sản phẩm tên là System/R.
- **1986** -- IBM phát triển nguyên mẫu đầu tiên về Relation Database và được chuẩn hóa bởi ANSI. Relation Database đầu tiên được công bố là Relational Software và sau đó là Oracle.



■ Phân loại ngôn ngữ SQL

- Ngôn ngữ DQL (Data Query Language)
- Ngôn ngữ DDL (Data Definition Language)
- Ngôn ngữ DML (Data Manipulation Language)
- Ngôn ngữ DCL (Data Control Language)



■ DQL (Data Query Language) – Ngôn ngữ truy vấn dữ liệu

- SELECT: Lấy các bản ghi cụ thể từ một hoặc nhiều bảng.



■ DDL (Data Definition Language) – Ngôn ngữ định nghĩa dữ liệu

- CREATE: Tạo một bảng, một View của bảng, hoặc đối tượng khác trong Database.
- ALTER: Sửa đổi một đối tượng Database đang tồn tại, ví dụ như một bảng.
- DROP: Xóa toàn bộ một bảng, một View của bảng hoặc đối tượng khác trong một Database.



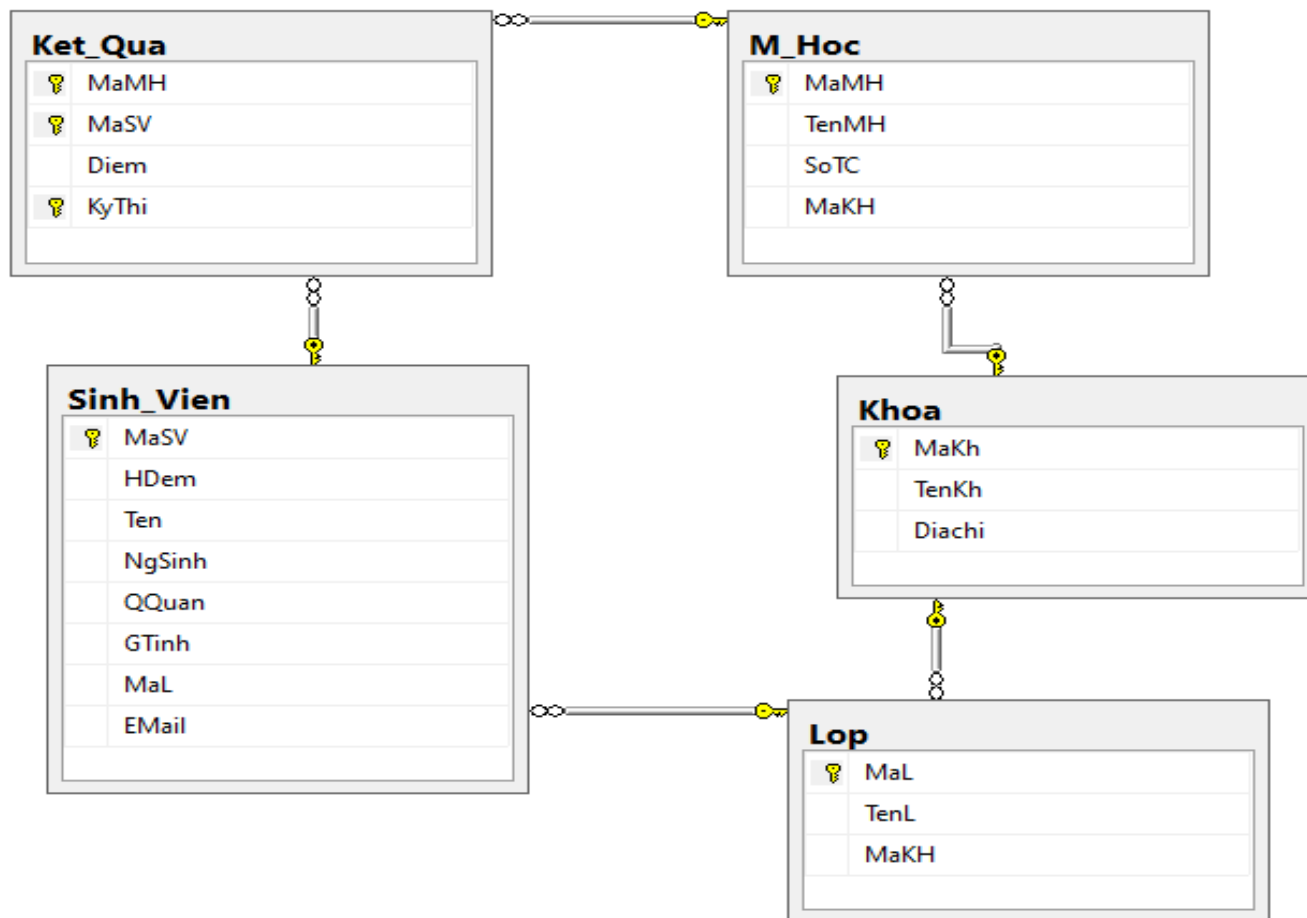
■ DML (Data Manipulation Language) – Ngôn ngữ thao tác dữ liệu

- INSERT: Tạo một bản ghi.
- UPDATE: Sửa đổi các bản ghi.
- DELETE: Xóa các bản ghi.



- DCL (Data Control Language) – Ngôn ngữ điều khiển dữ liệu
 - GRANT: Trao một quyền tới người dùng.
 - REVOKE: Thu hồi quyền đã trao cho người dùng.

2. Truy vấn dữ liệu từ bảng đơn





DQL

(DATA QUERY LANGUAGE)

2. Truy vấn dữ liệu từ bảng đơn



- SQL không phân biệt chữ hoa chữ thường
- Câu lệnh SQL có thể viết trên một hoặc nhiều dòng
- Các dấu tab và xuống dòng thường được sử dụng cho mục đích dễ đọc



Cú pháp tổng quát:

```
SELECT [*]/[ALL | DISTINCT] danh_sách_cột  
[INTO [tên_bảng_mới]]  
FROM {tên_bảng | tên_view}  
[WHERE điều_kiện_1]  
[GROUP BY danh_sách_cột_1]  
[HAVING điều_kiện_2]  
[ORDER BY danh_sách_cột_2 [ASC | DESC]]
```

2. Truy vấn dữ liệu từ bảng đơn



Danh sách chọn trong câu lệnh Select

Điều kiện truy vấn dữ liệu

Sắp xếp dữ liệu

Sử dụng bí danh

2. Truy vấn dữ liệu từ bảng đơn



- **Chọn tất cả các cột:** dùng dấu * trong câu lệnh Select
- **Ví dụ:** Cho biết thông tin của tất cả các lớp trong bảng lớp

```
Select * From Lop;
```

2. Truy vấn dữ liệu từ bảng đơn



- **Chọn các cột có giá trị khác nhau:** SQL sử dụng từ khóa **DISTINCT** để lấy các giá trị khác nhau trong các cột được chọn.
- Ví dụ: Cho biết đã có những môn nào được thi

```
Select distinct MaMH From Ket_Qua
```

2. Truy vấn dữ liệu từ bảng đơn



- *Chọn một số cột cụ thể*
- Ví dụ: Cho biết mã sinh viên, họ đệm, tên, giới tính của các sinh viên trong danh sách sinh viên

```
Select MaSV, Hdem, Ten, GioiTinh  
From Sinh_Vien
```



- **Hằng và biểu thức trong danh sách chọn:**
- **Biểu thức có thể chứa các phép toán số học và các hàm thư viện: count(); min(); max(); sum(); avg(); month(); year()...**
- **Ví dụ:**

```
Select Masv, HDem, Ten, 'Tuoi' = YEAR(GETDATE()) -  
YEAR(NgSinh) From Sinh_Vien
```

2. Truy vấn dữ liệu từ bảng đơn



- Giới hạn số dòng trong kết quả truy vấn:
- Ví dụ: Lấy ra 5 môn học đầu tiên trong danh sách các môn học

```
Select Top 5 MaMH, TenMH From M_Hoc
```

- Trả về 10% số các môn học có trong bảng môn học:

```
Select Top 10 percent MaMH, TenMH  
From M_Hoc
```



- ***Tạo bảng mới lấy dữ liệu từ kết quả của câu lệnh `SELECT`***

- Ví dụ: Câu lệnh sau lấy dữ liệu từ bảng SINH VIÊN và tạo ra một bảng mới có tên là DSSV gồm các thuộc tính: MaSV và Họ tên

```
Select MaSV, HDem+' '+Ten as Ho_ten  
Into DSSV From Sinh_Vien
```

2. Truy vấn dữ liệu từ bảng đơn



■ Sử dụng bảng DSSV mới tạo

```
Select * From DSSV
```

100 % <

Results Messages

	MaSV	Ho_ten	tuoi
1	1	Nguyen Van An	27
2	2	Tran Viet Anh	27
3	4	Le Thi Huyen	28
4	5	Le Huyen	28
5	6	Tran Le Khanh	28
6	7	Hoang Hai Ha	28
7	8	Nguyen Tuan Thanh	29



- Mệnh đề **WHERE** trong câu lệnh **SELECT** được sử dụng nhằm xác định các điều kiện truy xuất dữ liệu
- Ví dụ: Đưa ra danh sách gồm các sinh viên có giới tính là nam

```
Select MaSV, HDem, Ten, GTinh
```

```
From Sinh_Vien Where GTinh = 'nam'
```



Các toán tử được sử dụng trong các biểu thức điều kiện:

- Toán tử so sánh: =; >; >; <; <; <>; !>; !<
- Toán tử logic để kết hợp các điều kiện: AND/OR
- Các toán tử đặc biệt:
 - BETWEEN
 - IS NULL
 - LIKE
 - IN



- **Sử dụng *BETWEEN* để kiểm tra giới hạn của dữ liệu:** Để kiểm tra xem giá trị dữ liệu nằm trong (ngoài) một khoảng nào đó.
- **Ví dụ:** Trả về danh sách kết quả có điểm thi nằm trong khoảng từ 5 đến 10.

```
Select * From Ket_Qua  
Where Diem BETWEEN 5 AND 10
```



- **Giá trị NULL**
- Ví dụ: Kết quả của câu lệnh sau trả về danh sách các sinh viên có địa chỉ email là null.

Select * From Sinh_Vien

Where EMail is null;

Results		Messages						
	MaSV	HDem	Ten	NgSinh	QQuan	GTinh	MaL	EMail
1	2	Tran Viet	Anh	1993-12-12	Hai Duong	Nam	CNTTAK14	NULL
2	3	Le Thanh	Huong	1992-02-10	Hai Duong	Nam	CNTTBK14	NULL
3	8	Nguyen Tuan	Thanh	1991-07-22	Nghe An	Nam	CNTTBK13	NULL



■ Từ khoá *LIKE*

■ Các kí tự đại diện:

%	<i>Chuỗi ký tự bất kỳ gồm không hoặc nhiều ký tự</i>
-	<i>Ký tự đơn bất kỳ</i>
[]	<i>Ký tự đơn bất kỳ trong giới hạn được chỉ định (ví dụ[a-f]) hay một tập (ví dụ [abcdef])</i>
[^]	<i>Ký tự đơn bất kỳ không nằm trong giới hạn được chỉ định (ví dụ [^a-f] hay một tập (ví dụ [^abcdef])</i>

■ Ví dụ: Sinh viên có tên bắt đầu bởi chữ A và chữ B

```
Select * from Sinh_Vien Where Ten LIKE '[A,B]%'
```



Danh sách IN

■ Ví dụ:

```
Select * From Ket_Qua
```

```
Where MaMH in ( 'CSDL1' , 'TDC' , 'ToanRR' )
```

- Câu lệnh trên trả về danh sách các môn học là môn CSDL1 hoặc môn TDC hoặc môn ToanRR thay cho câu lệnh:

```
Select * From Ket_Qua
```

```
Where MaMH= 'CSDL1' OR MaMH= 'TDC' OR  
MaMH= 'ToanRR'
```



- Sử dụng hàm có sẵn trong truy vấn.

Tên_hàm ([danh_sách_tham_số])

- Một số hàm thông dụng

Chuỗi	Số	Thời gian	Gộp
LEN	FLOOR	DAY, MONTH, YEAR	COUNT
LOWER, UPPER	SQRT	DATEPART	SUM
LEFT	ROUND	DATENAME	MIN, MAX
RIGHT	RAND	GETDATE	SUM
SUBSTRING		DATEDIFF	AVG
RTRIM, LTRIM, TRIM		DATEADD	



- Cho biết danh sách gồm các thông tin: Mã sinh viên, họ tên và tuổi:

```
Select MaSV, Hdem, Ten as Ho_ten,  
tuoi = YEAR(getdate())-YEAR(NgSinh)  
From Sinh_Vien
```

- Cho biết tổng số sinh viên đã thi môn CSDL1:

```
Select COUNT(MaSV) From Ket_Qua  
Where MaMH='CSDL1'
```



■ Hàm ngày tháng:

<i>Function name</i>	<i>Parameters</i>	<i>Description</i>
<i>dateadd</i>	<i>(date part, number, date)</i>	<i>Adds the number of date parts to the date</i>
<i>datediff</i>	<i>(date part, date1, date2)</i>	<i>Calculates the number of date parts between two dates</i>
<i>Datename</i>	<i>(date part, date)</i>	<i>Returns date part from the listed date, as a character value (for example, October)</i>
<i>datepart</i>	<i>(date part, date)</i>	<i>Returns date part from the listed date as an integer</i>

2. Truy vấn dữ liệu từ bảng đơn



<i>Date part</i>	<i>Abbreviation</i>	<i>Values</i>
<i>year</i>	<i>yy, yyyy</i>	<i>1753-9999</i>
<i>quarter</i>	<i>qq, q</i>	<i>1-4</i>
<i>month</i>	<i>mm, m</i>	<i>1-12</i>
<i>day of year</i>	<i>dy, y</i>	<i>1-366</i>
<i>day</i>	<i>dd, d</i>	<i>1-31</i>
<i>week</i>	<i>wk, ww</i>	<i>0-51</i>
<i>weekday</i>	<i>dw</i>	<i>1-7(1 is Sunday)</i>
<i>hour</i>	<i>hh</i>	<i>(0-23)</i>
<i>minute</i>	<i>mi, n</i>	<i>(0-59)</i>
<i>second</i>	<i>ss, s</i>	<i>0-59</i>

2. Truy vấn dữ liệu từ bảng đơn



Ví dụ: Cho biết danh sách gồm họ tên và tháng sinh của mỗi sinh viên

```
Select HDem, Ten, DATENAME (MM, NgSinh) as Thang  
From Sinh_Vien
```

Kết quả:

	HDem	Ten	Thang
1	Nguyen Van	An	October
2	Tran Viet	Anh	October
3	Tran Le	Khanh	May
4	Hoang Hai	Ha	February
5	Hoang Hai	Ha	February
6	Nguyen Tuan	Thanh	July
7	Le	Huyen	February
8	Pham	Huyen	NULL



- **Sắp xếp kết quả truy vấn: ORDER BY ASC/DESC**
- Ví dụ: Sắp xếp danh sách kết quả theo chiều tăng dần của Mã sinh viên, sau đó điểm của mỗi sinh viên lại được sắp xếp điểm theo chiều giảm dần.

```
Select MaSV, MaMH, Diem From Ket_Qua Order  
by MaSV, Diem Desc;
```

- **Hoặc có thể viết:**

```
Select MaSV, MaMH, Diem From Ket_Qua Order  
by 1, 3 Desc
```

2. Truy vấn dữ liệu từ bảng đơn



■ *Đổi tiêu đề các cột*

```
Select 'Ma Sinh Vien'=MaSV, HDem+' '+Ten as  
Ho_ten, QQuan 'Que Quan' From Sinh_Vien;
```

Các cách thay đổi tiêu đề cột:

- Tiêu_đề_cột = tên_trường hoặc
- Tên_trường AS tiêu_đề_cột hoặc
- Tên_trường tiêu_đề_cột

	Ma Sinh Vien	Ho_ten	Que Quan
1	1	Nguyen Van An	Hai Phong
2	2	Tran Viet Anh	Hai Duong
3	3	Le Thanh Huong	Hai Duong
4	4	Le Thi Huyen	Ha Noi
5	5	Le Huyen	Ha Noi
6	6	Tran Le Khanh	Ha Noi
7	7	Hoang Hai Ha	NULL
8	8	Nguyen Tuan Thanh	Nghe An

- **Gom nhóm GROUP BY**
- **Được sử dụng sau Where & trước Order by (nếu có)**
 - ✓ Mệnh đề GROUP BY được sử dụng để tạo hiệu quả sắp xếp và tính toán theo từng phân nhóm.
 - ✓ Mệnh đề HAVING dùng để đặt điều kiện lọc cho phân nhóm con ở Group by.

3. Sử dụng hàm nhóm và mệnh đề HAVING

DEPTNO	JOB	SAL
10	MANAGER	2450
10	PRESIDENT	5000
10	CLERK	1300
20	CLERK	800
20	CLERK	1100
20	ANALYST	3000
20	ANALYST	3000
20	MANAGER	2975
30	SALESMAN	1600
30	MANAGER	2850
30	SALESMAN	1250
30	CLERK	950
30	SALESMAN	1500
30	SALESMAN	1250

“sum salaries in the EMP table for each job, grouped by department”

DEPTNO	JOB	SUM (SAL)
10	CLERK	1300
10	MANAGER	2450
10	PRESIDENT	5000
20	ANALYST	6000
20	CLERK	1900
20	MANAGER	2975
30	CLERK	950
30	MANAGER	2850
30	SALESMAN	5600

```
SELECT      [column,] group_function(column)
FROM        table
[WHERE      condition]
[GROUP BY   column]
[ORDER BY   column];
```

- Có thể sử dụng hàm AVG và SUM cho dữ liệu kiểu số
- Có thể sử dụng hàm Min và Max cho bất kì kiểu dữ liệu nào
- COUNT (*) trả về số hàng của bảng
- COUNT(TÊN CỘT) bỏ qua dòng mà giá trị cột đó là NULL

```
SELECT      column, group_function(column)
FROM        table
[WHERE      condition]
[GROUP BY   group_by_expression]
[ORDER BY   column] ;
```

- Chia các hàng trong kết quả tìm kiếm thành các nhóm nhỏ bởi mệnh đề **GROUP BY**
- Tất cả các cột trong danh sách không có trong hàm nhóm phải nằm trong mệnh đề **GROUP BY**
- Cột trong mệnh đề **GROUP BY** không nhất thiết phải có trong **SELECT**
- Có thể nhóm trên nhiều cột

- Cho biết mỗi môn học đã có bao nhiêu sinh viên có kết quả?

```
Select MaMH, COUNT(MaSV) as TongSo From Ket_Qua
Group by MaMH
```

- Tạo nhóm dữ liệu:

```
Select MaSV, MaMH
From Ket_Qua Group by MaSV, MaMH
```

	MaSV	MaMH
1	1	CSDL1
2	2	CSDL1
3	3	CSDL1
4	6	CSDL1
5	1	TA1
6	2	TA1
7	4	TA2
8	1	TDC
9	2	TDC
10	3	TDC
11	6	TDC

3. Sử dụng hàm nhóm và mệnh đề HAVING

EMP

DEPTNO	SAL
10	2450
10	5000
10	1300
20	800
20	1100
20	3000
20	3000
20	2975
30	1600
30	2850
30	1250
30	950
30	1500
30	1250

2916.6667

2175

1566.6667

“average
salary
in EMP
table
for each
department”

DEPTNO	AVG (SAL)
10	2916.6667
20	2175
30	1566.6667

■ Chú ý: Các câu lệnh không thể sử dụng

- Cột không có trong hàm nhóm & group by:

```
Select GTinh, count(MaSV) From Sinh_Vien
```

 Messages

Msg 8120, Level 16, State 1, Line 1

Column 'Sinh_Vien.GTinh' is invalid in the select list

because it is not contained in either an aggregate function or the GROUP BY clause.

- Không sử dụng mệnh đề WHERE để giới hạn điều kiện trên hàm nhóm (phải sử dụng having):

```
Select MaMH, COUNT(MaSV) as TongSo
```

 Messages

Msg 147, Level 15, State 1, Line 1

An aggregate may not appear in the WHERE clause unless it is in a subquery contained in a HAVING clause or a select list, and the column being aggregated is an outer reference.

3. Sử dụng hàm nhóm và mệnh đề HAVING

EMP

DEPTNO	SAL
-----	-----
10	2450
10	5000
10	1300
20	800
20	1100
20	3000
20	3000
20	2975
30	1600
30	2850
30	1250
30	950
30	1500
30	1250

5000

3000

2850

“maximum
salary
per department
greater than
\$2900”

DEPTNO	MAX (SAL)
-----	-----
10	5000
20	3000

- Các môn học có số lượng sinh viên thi lớn hơn 1:

```
Select MaMH, COUNT(MaSV) as TongSo From Ket_Qua  
Group by MaMH Having COUNT(MaSV)>1
```

- Cho danh sách các khoa có ít hơn hoặc bằng 3
lớp

```
Select MaKh, COUNT(MaL) so_lop From Lop  
Group by MaKH  
Having COUNT(MaL)<=3
```



1. Cho biết danh sách sinh viên sinh năm 1993
2. Cho biết danh sách sinh viên lớp CNTT BK14
3. Cho biết mã sinh viên của những sinh viên đã thi môn CSDL1 hoặc CSLT1
4. Đưa ra danh sách các sinh viên có họ là Lê
5. Cho biết điểm cao nhất của mỗi môn thi
6. Cho biết số lượng sinh viên của mỗi lớp
7. Cho danh sách các lớp có ít hơn 10 sinh viên
8. Đếm số lượng sinh viên theo giới tính



■ Các loại nối

- Nối trong
- Nối ngoài
- Nối chéo
- Nối bằng nhau
- Tự nối



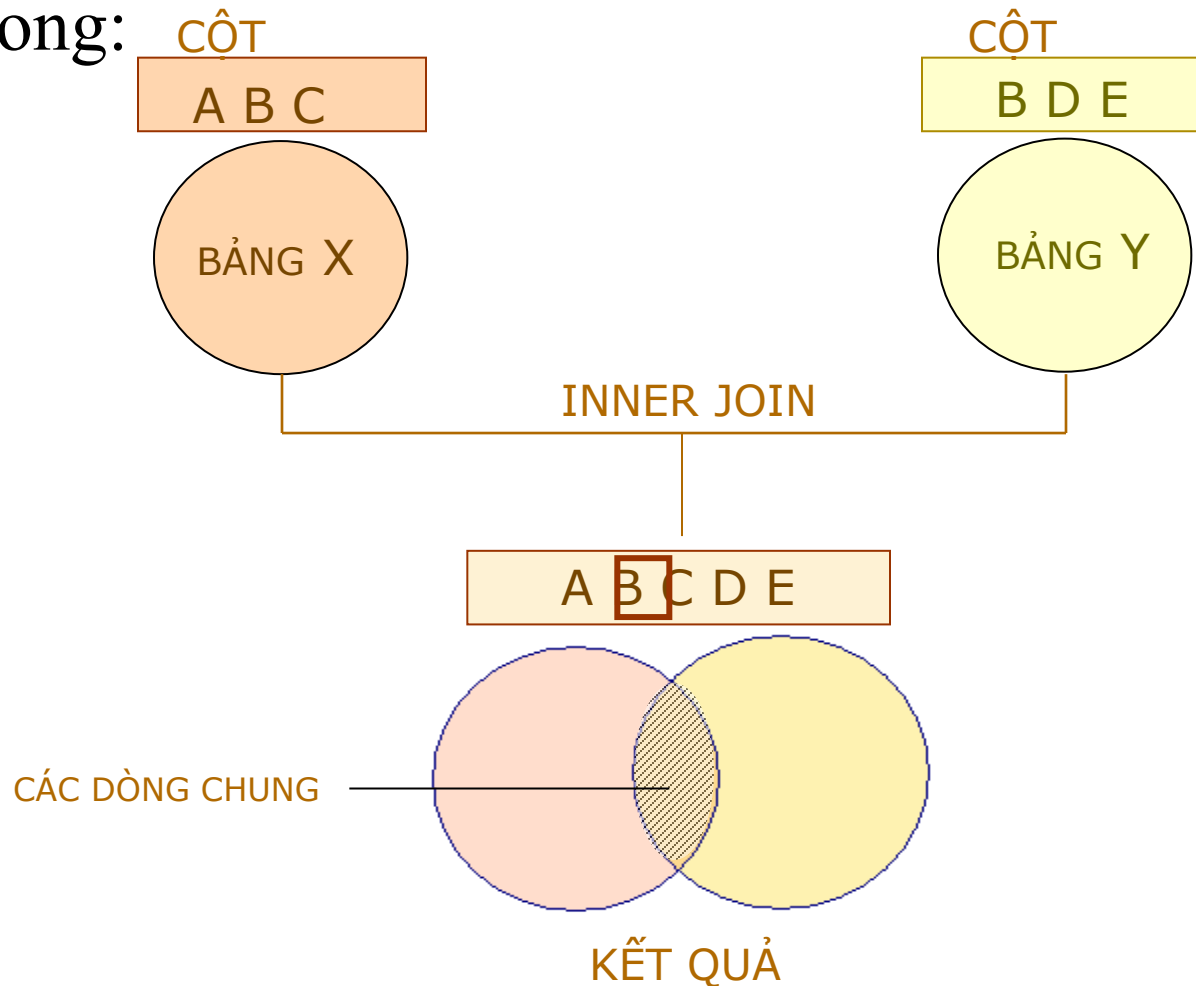
- **Phép nối trong (INNER JOIN)**
- Thực hiện nối nhiều bảng, trong đó các giá trị thỏa mãn điều kiện kết nối mới được hiển thị trong kết quả truy vấn
- Cú pháp

```
SELECT danh_sách_cột  
FROM bảng_1 JOIN/INNER JOIN bảng_2  
ON bảng_1.tên_cột SO_SÁNH  
bảng_2.tên_cột
```

4. Truy vấn dữ liệu từ nhiều bảng



Nối trong:



4. Truy vấn dữ liệu từ nhiều bảng



- **Ví dụ:** Cho danh sách thông tin gồm MaSV, Hdem, Ten, MaMH, Diem

```
Select Sinh_Vien.MaSV, HDem, Ten, MaMH, Diem  
From Sinh_Vien Inner Join Ket_Qua On  
Sinh_Vien.MaSV = Ket_Qua.MaSV
```

- **Kết quả**

Results		Messages			
	MaSV	HDem	Ten	MaMH	Diem
1	1	Nguyen Van	An	CSDL1	9
2	2	Tran Viet	Anh	CSDL1	9
3	3	Le Thanh	Huong	CSDL1	4
4	6	Tran Le	Khanh	CSDL1	7
5	1	Nguyen Van	An	TA1	9
6	2	Tran Viet	Anh	TA1	7
7	4	Le Thi	Huyen	TA2	9
8	1	Nguyen Van	An	TDC	8
9	2	Tran Viet	Anh	TDC	4
10	2	Tran Viet	Anh	TDC	8



- ***Phép nối ngoài (OUTER JOIN)***
- Kết quả trả về bao gồm tất cả bản ghi của một bảng và mỗi bản ghi kết nối với bảng còn lại
- Hiển thị giá trị NULL cho những ô không thỏa mãn điều kiện nối



■ Có 3 loại nối ngoài:

- Nối ngoài trái (left outer join/left join)
- Nối ngoài phải (Right outer Join/ Right Join)
- Nối ngoài hai phía (Full outer join/Full join)

■ Cú pháp:

SELECT danh_sách_cột

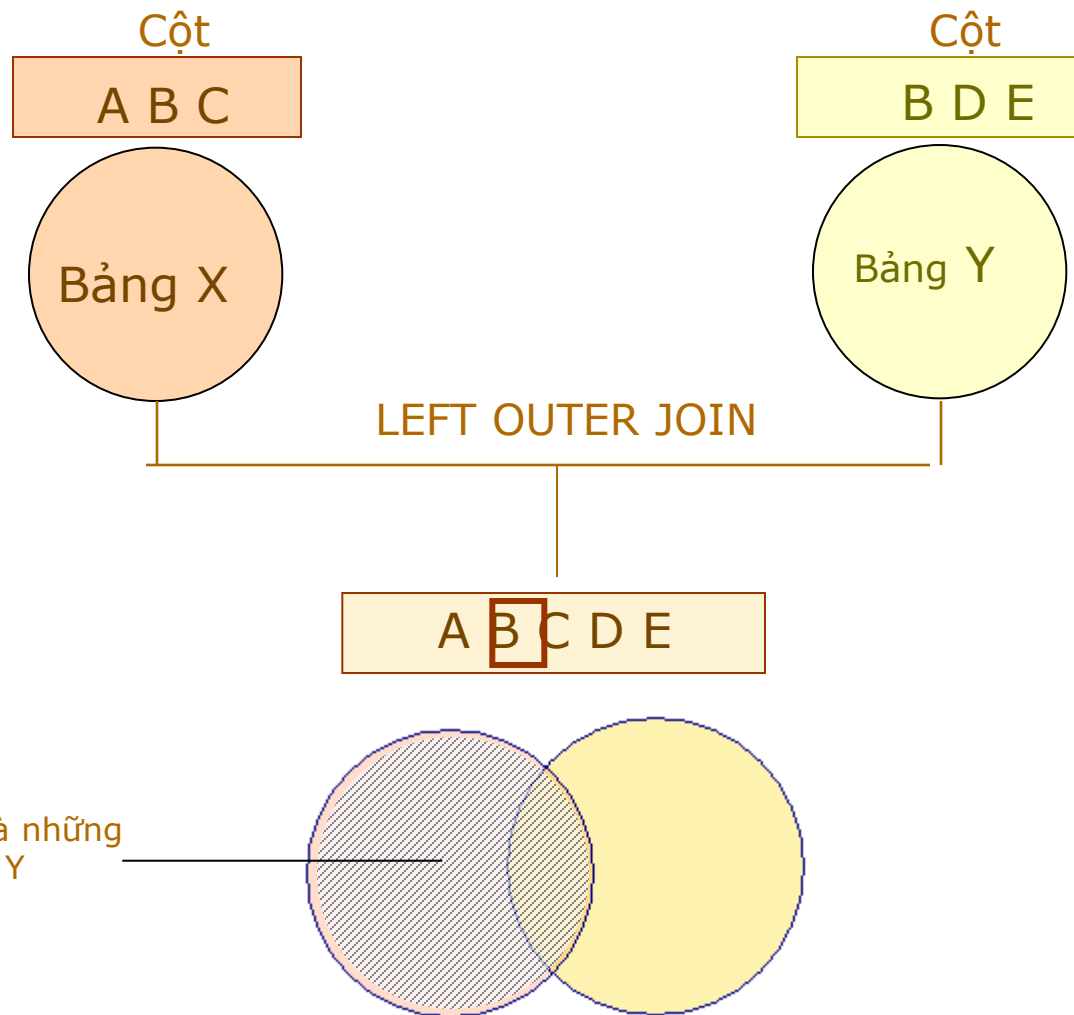
FROM bảng_1 [**LEFT** | **RIGHT** | **FULL**] **OUTER JOIN**

bảng_2

ON bảng_1.tên_cột **SO_SÁNH** bảng_2.tên_cột



Nối ngoài trái:



Tất cả bản ghi từ bảng X và những bản ghi giống nhau ở bảng Y

4. Truy vấn dữ liệu từ nhiều bảng



Ví dụ:

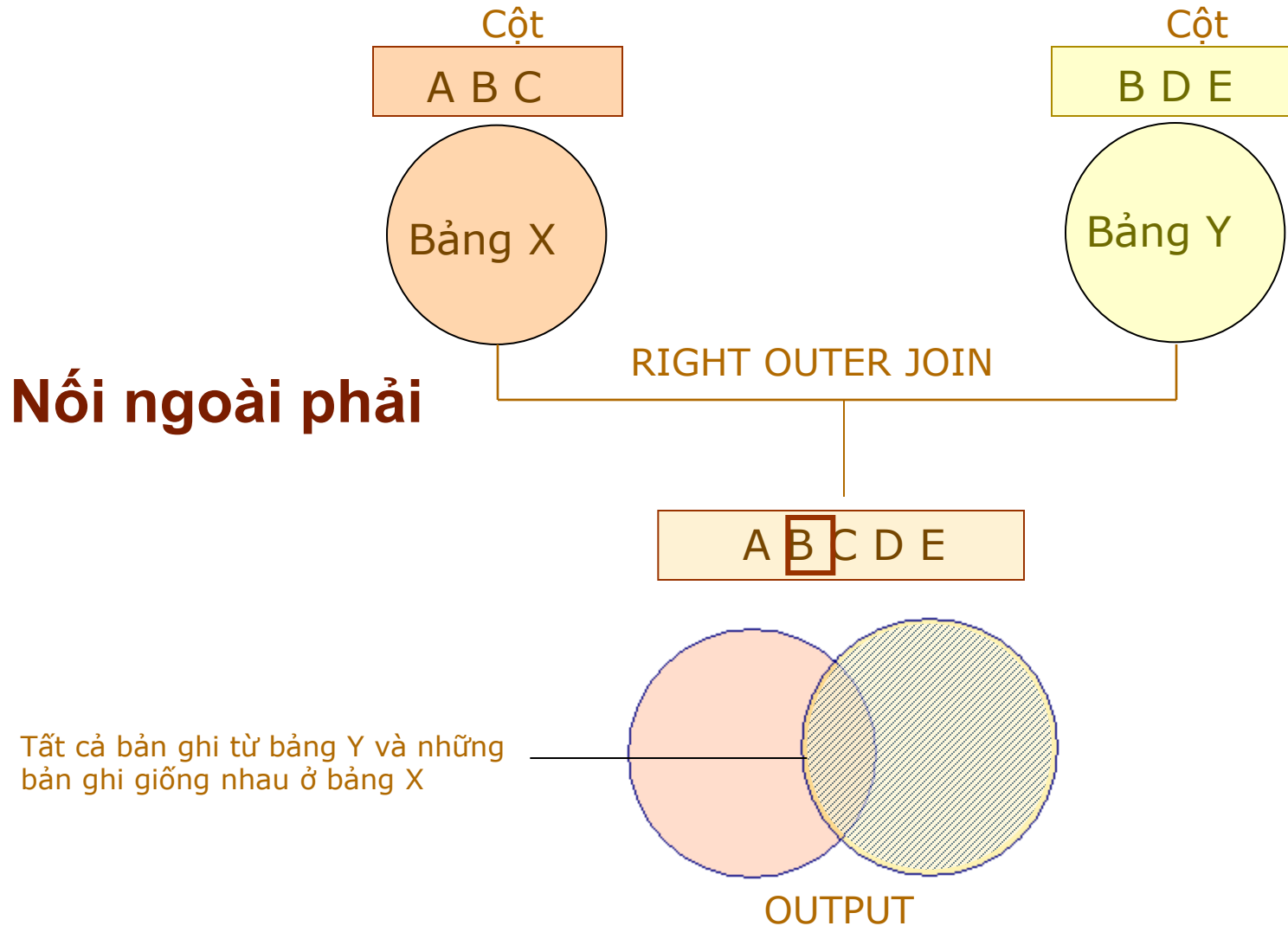
```

Select Sinh_Vien.MaSV, HDem, Ten, MaMH, Diem
From Sinh_Vien left outer join Ket_Qua On
Sinh_Vien.MaSV = Ket_Qua.MaSV;
    
```

■ Kết quả

	MaSV	HDem	Ten	MaMH	Diem
7	2	Tran Viet	Anh	TDC	8
8	2	Tran Viet	Anh	TDNH1	6
9	3	Le Thanh	Huong	CSDL1	4
10	3	Le Thanh	Huong	TDC	8
11	4	Le Thi	Huyen	TA2	9
12	5	Le	Huyen	NULL	NULL
13	6	Tran Le	Khanh	CSDL1	7
14	6	Tran Le	Khanh	TDC	0
15	6	Tran Le	Khanh	TDC	0
16	7	Hoang Hai	Ha	NULL	NULL
17	8	Nguyen T...	Thanh	NULL	NULL

4. Truy vấn dữ liệu từ nhiều bảng



4. Truy vấn dữ liệu từ nhiều bảng



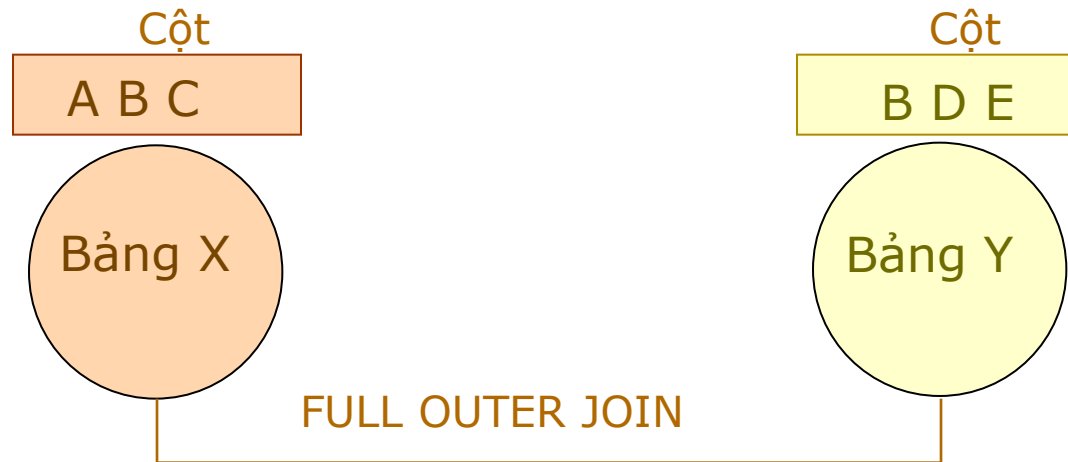
■ Ví dụ:

Select MaSV, Hdem, Ten, Lop.MaL, TenL **From**
Sinh_Vien **Right outer Join** Lop **On**
Sinh_Vien.MaL = Lop.MaL

■ Kết quả

	MaSV	Hdem	Ten	MaL	TenL
1	NULL	NULL	NULL	CNTTAK13	Cong nghe Thông tin A K13
2	1	Nguyen Van	An	CNTTAK14	Cong nghe Thông tin A K14
3	2	Tran Viet	Anh	CNTTAK14	Cong nghe Thông tin A K14
4	7	Hoang Hai	Ha	CNTTBK13	Cong nghe Thông tin B K13
5	8	Nguyen Tuan	Thanh	CNTTBK13	Cong nghe Thông tin B K13
6	3	Le Thanh	Huong	CNTTBK14	Cong nghe Thông tin B K14
7	NULL	NULL	NULL	TAAK14	Tieng Anh A K14
8	4	Le Thi	Huyen	TABK14	Tieng Anh B K14
9	5	Le	Huyen	TABK14	Tieng Anh B K14
10	6	Tran Le	Khanh	TABK14	Tieng Anh B K14
11	NULL	NULL	NULL	TCNHAK14	Tai chinh Ngân hàng A K14

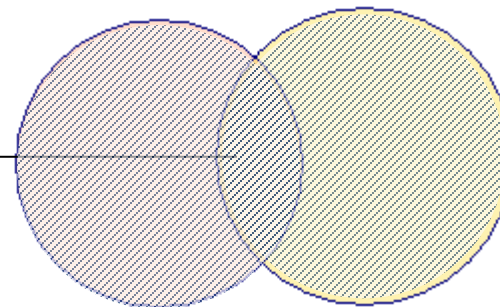
4. Truy vấn dữ liệu từ nhiều bảng



Nối ngoài hai phía



Tất cả bản ghi từ bảng X và bảng Y.
Bản ghi giống nhau chỉ xuất hiện 1 lần



OUTPUT

4. Truy vấn dữ liệu từ nhiều bảng



■ Ví dụ:

```
Select  Sinh_Vien.MaSV, HDem, Ten, MaMH, Diem
From  Sinh_Vien Full outer join Ket_Qua
On  Sinh_Vien.MaSV = Ket_Qua.MaSV
```

■ Kết quả

	MaSV	HDem	Ten	MaMH	Diem
8	2	Tran Viet	Anh	TDNH1	6
9	4	Le Thi	Huyen	TA2	9
10	5	Le	Huyen	NULL	NU...
11	6	Tran Le	Khanh	CSDL1	7
12	6	Tran Le	Khanh	TDC	0
13	6	Tran Le	Khanh	TDC	0
14	7	Hoang Hai	Ha	NULL	NU...
15	8	Nguyen T...	Thanh	NULL	NU...

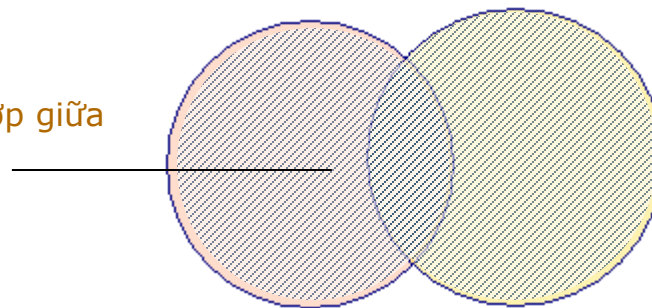
4. Truy vấn dữ liệu từ nhiều bảng



Nối chéo

A B C D E

Tất cả bản ghi ($n \times m$) kết hợp giữa bảng X và bảng Y



OUTPUT

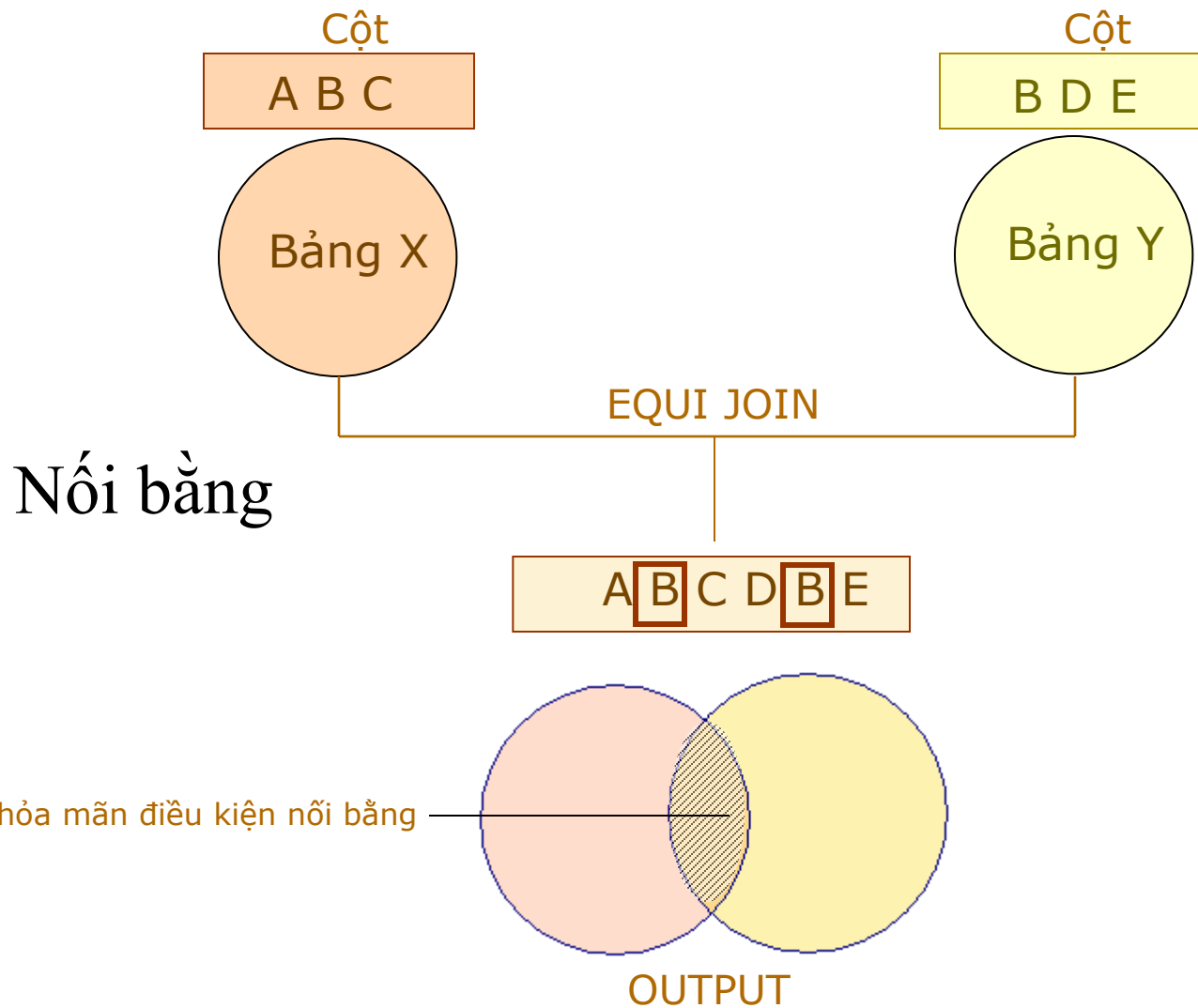
4. Truy vấn dữ liệu từ nhiều bảng



- Ví dụ: `Select p.MaMH, p.TenMH, p.SoTC, p1.MaSV, p1.Diem`
`From M_hoc p Cross Join Ket_qua p1`
- Kết quả

	MaMH	TenMH	SoTC	MaSV	Diem
1	CSDI1	Co so du lieu 1	2	6	8
2	CSDI1	Co so du lieu 1	2	6	5
3	CSDI1	Co so du lieu 1	2	8	7
4	CSDI1	Co so du lieu 1	2	9	9
5	CSDI1	Co so du lieu 1	2	9	6
6	CSDI1	Co so du lieu 1	2	10	8
7	CSDI1	Co so du lieu 1	2	11	4
8	CSDI2	Co so du lieu 2	2	6	9
...			...	...	

4. Truy vấn dữ liệu từ nhiều bảng





■ Ví dụ:

Select * From Sinh_Vien, Ket_Qua

Where Sinh_Vien.MaSV=Ket_Qua.MaSV

Results Messages												
	MaSV	HDem	Ten	NgSinh	QQuan	GTinh	MaL	EMail	MaMH	MaSV	Diem	KyThi
1	1	Nguyen Van	An	1993-10-12	Hai Phong	Nam	CNTTAK14	AnNv@gmail.com	CSDL1	1	9	3
2	2	Tran Viet	Anh	1993-12-12	Hai Duong	Nam	CNTTAK14	NULL	CSDL1	2	9	3
3	3	Le Thanh	Huong	1992-02-10	Hai Duong	Nam	CNTTBK14	NULL	CSDL1	3	4	3
4	6	Tran Le	Khanh	1992-05-20	Ha Noi	Nu	TABK14	Khanhtl@gmail.com	CSDL1	6	7	3
5	1	Nguyen Van	An	1993-10-12	Hai Phong	Nam	CNTTAK14	AnNv@gmail.com	TA1	1	9	1
6	2	Tran Viet	Anh	1993-12-12	Hai Duong	Nam	CNTTAK14	NULL	TA1	2	7	1
7	4	Le Thi	Huyen	1992-02-15	Ha Noi	Nu	TABK14	HuyenLT@gmail.com	TA2	4	9	3
8	1	Nguyen Van	An	1993-10-12	Hai Phong	Nam	CNTTAK14	AnNv@gmail.com	TDC	1	8	1
9	2	Tran Viet	Anh	1993-12-12	Hai Duong	Nam	CNTTAK14	NULL	TDC	2	4	1
10	2	Tran Viet	Anh	1993-12-12	Hai Duong	Nam	CNTTAK14	NULL	TDC	2	8	3
11	3	Le Thanh	Huong	1992-02-10	Hai Duong	Nam	CNTTBK14	NULL	TDC	3	8	1



- **VD: Tìm thông tin của sinh viên trong lớp có cùng ngày sinh với sinh viên Nguyễn Văn An.**

```
Select b.MaSV, b.HDem, b.Ten, b.NgSinh, TenL
```

```
From Sinh_Vien a, Sinh_Vien b, Lop c
```

```
Where a.MaL=c.MaL AND a.HDem='Nguyen Van' AND
```

```
a.Ten='An' AND a.NgSinh= b.NgSinh AND
```

```
a.MaSV<>b.MaSV
```



- Ví dụ: Bảng kết quả cần các thông tin sau: Mã sinh viên, họ tên sinh viên, tên lớp và tên khoa của các sinh viên đó.

```
Select MaSV, HDem+' '+Ten as 'Ho va ten',  
TenL, TenKh From Sinh_Vien a join Lop b on  
a.MaL=b.MaL join Khoa c on b.MaKH=c.MaKh
```



■ Là gì?

- Đặt truy vấn bên trong một truy vấn khác

■ Dùng khi nào?

- Điều kiện truy vấn dữ liệu cần phải sử dụng đến kết quả của một truy vấn khác.



■ Lưu ý:

- Một truy vấn con phải được viết trong cặp dấu ngoặc
- ORDER BY không được phép sử dụng trong truy vấn con
- Một truy vấn con thường được sử dụng làm điều kiện trong mệnh đề WHERE hoặc HAVING của một truy vấn khác.



■ *Sử dụng với ALL/ ANY*

- ALL được sử dụng khi cần so sánh giá trị của biểu thức với tất cả các giá trị trả về trong kết quả của truy vấn con
- ANY có kết quả đúng khi chỉ cần một giá trị bất kỳ nào đó trong kết quả của truy vấn con thoả mãn điều kiện



■ Ví dụ:

```
Select * From Ket_qua Where diem>ALL  
(select Diem From ket_qua Where Masv=3)
```

■ Khác như thế nào với:

```
Select * From Ket_qua Where diem>ANY  
(select Diem From ket_qua Where Masv=3)
```



- *Sử dụng với IN/ NOT IN*

- Ví dụ: Tìm những sinh viên có cùng ngày sinh với sinh viên Nguyễn Văn An

```
Select MaSV, HDem, Ten, NgSinh, TenL
From Sinh_Vien a JOIN Lop b on a.MaL=b.MaL
Where HDem<>'Nguyen Van' AND Ten<>'An' AND
NgSinh in (Select NgSinh From Sinh_Vien
Where HDem='Nguyen Van' AND Ten='An' )
```



- *Sử dụng với EXISTS/ NOT EXISTS:* để kiểm tra xem một truy vấn con có trả về dòng kết quả nào hay không
- Ví dụ: Cho danh sách các sinh viên chưa thi môn nào

```
Select * From Sinh_Vien
```

```
Where not exists (Select * From Ket_Qua Where  
Sinh_Vien.MaSV=Ket_Qua.MaSV)
```

- Điểm khác biệt là trong danh sách chọn của truy vấn con có thể có nhiều hơn hai cột.



1. Cho biết thông tin Khoa của lớp CNTTAK14
2. Cho biết thông tin của sinh viên có điểm thi cao nhất
3. Cho biết thông tin các sinh viên có kết quả thi môn CSDL1
4. Cho biết danh sách thông tin gồm: MaSV, Hdem, Ten, TenL, TenKh
5. Bảng điểm của sinh viên có mã số 2
6. Cho biết các sinh viên chưa thi môn nào
7. Cho biết tên lớp của những sinh viên đã thi môn CSDL1
8. Cho biết số sinh viên đã thi môn CSDL1
9. Thống kê xem mỗi môn thi có bao nhiêu sinh viên dự thi
10. Cho biết điểm trung bình của mỗi sinh viên



■ Toán tử UNION

```
select_1 UNION [ALL] select_2 { [UNION [ALL] select_3] } ...
```

- ALL được sử dụng nếu muốn biểu diễn tất cả các dòng (kể cả trùng lặp) trong danh sách kết quả.

■ Hợp hai bảng là một bảng mới gồm tất cả các hàng xuất hiện trong một hoặc cả hai bảng.

■ Ví dụ:

```
Select MaL From Sinh_Vien  
union  
Select MaL from Lop
```



DDL

(DATA DEFINITION LANGUAGE)



■ Các đối tượng:

- Cơ sở dữ liệu
- Bảng
- Ràng buộc



Mỗi CSDL chứa trong 3 kiểu tập tin

- Primary data file (.mdf): bắt buộc có dùng để lưu trữ thông tin liên quan đến cấu trúc và đặc điểm của chính database đó.
- Secondary data file (.ndf): dùng để lưu trữ các đối tượng dữ liệu không nằm trong tập tin dữ liệu chính. Loại tập tin này không bắt buộc phải có khi tạo mới CSDL.
- Transaction log file (.ldf): dùng để lưu vết các giao tác, là những hành động dùng cập nhật dữ liệu (thêm, sửa, xóa) vào các bảng do người tác động trên CSDL



■ Tạo cơ sở dữ liệu

```
CREATE DATABASE database_name  
  
[ ON [ PRIMARY ] [ < filespec > ] ]  
[ LOG ON { < filespec > [ ,...n ] } ]  
< filespec > ::=  
  
( [ NAME = logical_file_name , ]  
FILENAME = 'os_file_name'  
[ , SIZE = size ]  
[ , MAXSIZE = { max_size | UNLIMITED } ]  
[ , FILEGROWTH = growth_increment ] )  
[ ,...n ]
```

■ Xóa CSDL: DROP DATABASE <Ten CSDL>



■ *Tạo CSDL QL_SinhVien*

```
create database QL_SinhVien
ON (Name = ql_sv_data,
    FileName = 'd:\tempt\ql-sv.mdf',
    Size = 2mb,
    FileGrowth = 10%
)
Log On
(Name = qlsv_log,
filename = 'd:\tempt\qlsvlog.ldf',
size = 2mb,
maxsize = unlimited)
```



■ *Sửa CSDL*

```
ALTER DATABASE <tên CSDL>  
[ADD FILE <tên tệp .MDF, .NDF> [, ...]  
    [ TO FILEGROUP <Tên File Group>] ]  
[ADD LOG FILE < tên tệp .LDF> [, ...]]  
[ADD FILEGROUP <Tên File Group>]  
[REMOVE FILE <Tên logic của tệp >]  
[REMOVE FILEGROUP <Tên File Group>]  
[MODIFY NAME = <Tên mới cho Database>]  
[MODIFY FILE < tên tệp>]  
[MODIFY FILEGROUP <Tên File Group> <Thuộc tính  
File Group>|NAME = <Tên mới>]  
[COLLATE <Tên collation>]
```



Thêm một file secondary vào CSDL QL_SinhVien

```
Alter database QL_SinhVien
Add file
(
    name = QLSV_new,
    filename = 'D:\tempt\sv_new.ndf',
    size = 3mb,
    maxsize = 5mb,
    filegrowth = 0%
)
```

Sửa file

```
Alter database QL_SinhVien
Modify file (name = qlsv_log, size = 5 mb)
```



■ Tạo bảng

```
CREATE TABLE tên_bảng (  
    tên_cột_1 kiểu_dữ_liệu[option],,  
    tên_cột_2 kiểu_dữ_liệu[option],,  
    .....);
```

■ Thứ tự tạo bảng?

7. Ngôn ngữ định nghĩa dữ liệu



```
Create table Sinh_Vien
(
    MaSV int identity(1,1) primary key not null,
    HDem varchar(20) not null,
    Ten varchar (10) not null,
    NgSinh datetime,
    QQuan varchar(50),
    GTinh char(3),
    MaL char(10) not null,
    EMail char(25)
)

Create table Ket_Qua
(
    MaMH char(10) not null,
    MaSV int not null,
    Diem float,
    KyThi tinyint not null,
    Constraint PK_KQ primary key (MaMH, MaSV, KyThi)
)
```



■ Sửa bảng

```
ALTER TABLE <Tên bảng> ADD <tên cột> | ALTER  
COLUMN <Tên cột> <Kiểu dữ liệu> [<Các thuộc tính khác>]  
| DROP COLUMN <Tên cột> | ADD CONSTRAINT <Tên  
ràng buộc> <Định nghĩa ràng buộc> | DROP CONSTRAINT  
<Tên ràng buộc> | ADD PRIMARY KEY <Tên cột> | ADD  
FOREIGN KEY <Tên cột> REFERENCES <Tên bảng>
```

■ Ví dụ:

```
Alter table M_Hoc Alter column TenMH  
nvarchar(30) null
```



- **Xóa bảng: Thứ tự xóa bảng?**

DROP TABLE Ten_Bang

- **Ví dụ:**

Drop table SINH_VIEN



- **Tạo các bảng trong CSDL mẫu**
- **Thêm bảng Giảng viên với các quan hệ: Giảng viên thuộc Khoa, Giảng viên giảng dạy các môn học (mỗi môn học có nhiều Giảng viên giảng dạy)**



■ Có thể tạo ràng buộc khi:

- CREATE TABLE
- ALTER TABLE

■ Cú pháp:

```
CREATE TABLE tên_bảng  
(  
    tên_cột kiểu_DL CONSTRAINT tên_constraint  
    loại_constraint [,CONSTRAINT  
        tên_constraint  
        loại_constraint],  
    ...  
)
```



■ Các loại ràng buộc

- Khóa chính
- Khóa ngoại
- Duy nhất
- Giá trị mặc định
- Check



- **Khóa chính: không chấp nhận giá trị NULL**

- **Cú pháp:**

```
[CONSTRAINT tên_ràng_buộc] PRIMARY KEY [(danh_sách_cột)]
```



■ Cách 1:

- *CREATE TABLE Ten_Bang (Cot_1 KieuDL NOT NULL PRIMARY KEY, Cot_2 KieuDL..)*

■ Cách 2:

- *CREATE TABLE Ten_Bang (Cot_1 KieuDL, Cot_2 KieuDL.., CONSTRAINT PK_TEN PRIMARY KEY(Cot_1, Cot_2..))*

■ Cách 3:

- *ALTER TABLE Ten_Bang ADD CONSTRAINT PK_Ten PRIMARY KEY(Cot_1, Cot_2..)*



Cách 1: Create table M_Hoc
(
 MaMH char(10) primary key not null,
 TenMH varchar(30),
 SoTC int,
 MaKH char(10) not null
)

Alter table M_Hoc Add constraint
PK_MH primary key (MaMH)

Cách 2: Create table M_Hoc
(
 MaMH char(10) not null,
 TenMH varchar(30),
 SoTC int,
 MaKH char(10) not null,
 Constraint PK_MH primary key (MaMH)
)



■ *Khóa ngoại*

```
[CONSTRAINT tên_ràng_buộc] FOREIGN KEY  
[(danh_sách_cột)] REFERENCES tên_bảng_tham_chiếu  
[(danh_sách_cột_tham_chiếu)] [ON DELETE CASCADE | NO  
ACTION | SET NULL | SET DEFAULT] [ON UPDATE  
CASCADE | NO ACTION | SET NULL | SET DEFAULT]
```

SQL chuẩn đưa ra 4 cách xử lý khi **Delete** hoặc **Update**:

CASCADE: Tự động xóa (cập nhật) nếu bản ghi được tham chiếu bị xóa (cập nhật).

NO ACTION: (Mặc định) Nếu bản ghi trong bảng tham chiếu đang được tham chiếu bởi một bản ghi bất kỳ trong bảng được định nghĩa thì bản ghi đó không được phép xóa hoặc cập nhật (đối với cột được tham chiếu).

SET NULL: Cập nhật lại khoá ngoài của bản ghi thành giá trị NULL (nếu cột cho phép nhận giá trị NULL).

SET DEFAULT: Cập nhật lại khoá ngoài của bản ghi nhận giá trị mặc định (nếu cột có qui định giá trị mặc định).



■ Tạo khóa ngoại trong bảng lớp

Create table Lop

```
(MaL char(10) primary key not null,  
  TenL varchar(30),  
  MaKH char(10)  
  Constraint FK_Lop_Khoa Foreign key (MaKH)  
references Khoa  
)
```

■ Hoặc:

Alter table Lop

```
Add constraint FK_Lop_Khoa Foreign key (MaKH)  
references Khoa
```



■ *Duy nhất*

```
[CONSTRAINT tên_ràng_buộc] UNIQUE [(danh_sách_cột)]
```

Alter table Lop

Add constraint UC_Lop Unique (TenL, MaKH);

■ *Giá trị mặc định*

```
[CONSTRAINT tên_ràng_buộc] Default [(danh_sách_cột)]
```

Alter table ket_qua

Add constraint DF_KQ Default 0 for Diem



CHECK:

- Ràng buộc CHECK được dùng để hạn chế những dữ liệu lưu trong một trường.

```
[CONSTRAINT tên_ràng_buộc] CHECK (<điều kiện>)
```

- Ví dụ:

```
Alter table sinh_vien
```

```
Add constraint CC_SV Check (GTinh = 'Nam' or  
GTinh = 'Nu')
```



Tạo các ràng buộc sau:

- Khóa chính, khóa ngoại cho các bảng
- Thuộc tính Số tín chỉ trong bảng Môn học phải lớn hơn 0 và nhỏ hơn 10
- Thuộc tính Điểm thi phải lớn hơn 0 và nhỏ hơn 10, mặc định nhận giá trị 0 khi không nhập giá trị
- Thuộc tính Giới tính chỉ nhận giá trị Nam hoặc Nữ
- Thuộc tính Tên Khoa, Tên môn học là duy nhất



DML

(DATA MANIPULATION LANGUAGE)



■ Thêm mới

- INSERT
- SELECT...INTO

■ Sửa

- UPDATE

■ Xóa

- DELETE
- TRUNCATE



■ Thêm mới bản ghi:

```
INSERT [INTO] {tên_bảng} [(danh_sách_cột)]  
VALUES (DEFAULT | danh_sách_giá_trị | Lệnh_truy_vấn)
```

■ Ví dụ:

```
Insert into Khoa(MaKH, TenKH, DiaChi) Values  
( 'CNTT' , 'Cong nghe Thong tin' , '603_604' )
```

■ Bổ sung dữ liệu nhiều dòng từ bảng khác

```
Insert into Lop (MaL, TenL) Select MaL, TenL  
From DS_Lop
```



■ Ví dụ:

```
INSERT INTO universities.dbo.country (id,  
country_name) VALUES  
(1,'Argentina'),  
(2,'Australia'),  
(3,'Austria'),  
(4,'Bangladesh');
```



■ Cập nhật dữ liệu

UPDATE *tên_bảng*

SET *cột_1 = giá_trị_1* [, *cột_2 = giá_trị_2,...*]

[FROM *tên_bảng***]**

[WHERE *điều_kiện***]**

Ví dụ:

```
Update Sinh_Vien set EMail = 'huyenle@gmail.com'  
Where MaSV = 5
```



- **Xóa dữ liệu:**
- Lần lượt từng bản ghi

```
DELETE [FROM] tên_bảng  
[WHERE điều_kiện]
```

Ví dụ:

```
Delete From Sinh_Vien Where MaSV=5
```

- Toàn bộ dữ liệu trong bảng

```
TRUNCATE TABLE tên_bảng
```

Ví dụ:

```
Truncate Table Sinh_Vien
```



- **Nhập vào 20 dòng dữ liệu cho các bảng**
- **Cập nhật lại tên của các lớp trong khoa Công nghệ Thông tin (dạng: CNTT.Tên lớp)**



- Giả sử, cứ mỗi ngày bạn cần một danh sách các sản phẩm. Vậy bạn chọn cách gõ cùng một truy vấn vào mỗi cuối ngày? Hay một bảng dữ liệu lưu sẵn kết quả của truy vấn đó trong CSDL?

 View

- VIEW là là một BẢNG ảo được định nghĩa dựa trên truy vấn SELECT. Truy vấn có thể chứa các cột, biểu thức, bí danh, các hàm và lấy từ một hoặc nhiều bảng khác nhau (bảng cơ sở).



■ Mục đích:

- Giới hạn các cột và các dòng dữ liệu của bảng, đồng thời kiểm soát quyền truy cập vào bảng.
- Ẩn đi các truy vấn phức tạp, giúp đơn giản hóa việc sử dụng
- Hạn chế chèn/cập nhật trong những trường hợp nhất định



■ Tạo view:

CREATE VIEW *viewname* [*column_list*] AS SELECT *query*

■ Ví dụ:

**Create view DSlop_khoa as Select MaSV, HDem, Ten,
TenL, TenKh From Sinh_Vien a join Lop b on
a.MaL=b.MaL join Khoa c on b.MaKH=c.MaKh**



■ Ví dụ:

■ **Create View SoLuongThi (TenMH,TongSo) As Select**
TenMH, COUNT(MaSV) From Ket_Qua join M_Hoc on
Ket_Qua.MaMH=M_Hoc.MaMH Group by TenMH Having
COUNT(MaSV)>1



- Tương tự như đối với bảng, view cũng sử dụng Alter để sửa và Drop để xóa:

- **Ví dụ sửa view:**

```
Alter View SoLuongThi (TenMH,TongSo) As Select  
TenMH, COUNT(MaSV) From Ket_Qua join M_Hoc on  
Ket_Qua.MaMH=M_Hoc.MaMH Group by TenMH
```

- **Ví dụ xóa view:**

```
Drop view SoLuongThi
```



■ Truy vấn trên view:

- `Select * from DSlop_khoa`
- `Select TenKH, COUNT(MaSV) from DSlop_khoa group by TenKH`
- `Select * from SoLuongThi where TongSo>1`



- Một khung nhìn (view) có thể sử dụng các câu lệnh Insert/Update/Delete như một bảng. Khi đó, tùy từng trường hợp mà dữ liệu cũng sẽ được thêm/sửa/xóa ở bảng cơ sở.
- **Thêm dữ liệu trên view:**
 - Tạo view:
Create view DSSV
As select MaSV, HDem, Ten, MaL From Sinh_Vien
 - Thêm dữ liệu vào view
Insert into DSSV
values ('Dang Quang', 'Khanh', 'TAAK14')



■ Sửa dữ liệu trên view:

Update DSSV

set Ten = 'Thanh'

Where MaSV = 9

■ Xóa dữ liệu trên view:

Delete From DSSV Where MaSV = 9

Với các ví dụ trên, dữ liệu đều được thêm/sửa/xóa trên bảng cơ sở



■ **Dữ liệu sẽ không được thêm/sửa/xóa vào bảng khi thêm/sửa/xóa dữ liệu vào view nếu:**

- Mệnh đề FROM trong định nghĩa view bao gồm nhiều bảng
- Một cột của khung nhìn được lấy từ một hàm tổng hợp.
- Câu lệnh SELECT trong khung nhìn chứa mệnh đề GROUP BY hoặc tùy chọn DISTINCT.
- Không thể thêm/sửa nếu một cột của khung nhìn được dẫn xuất từ một hằng số hoặc một biểu thức.



■ Đặt vấn đề:

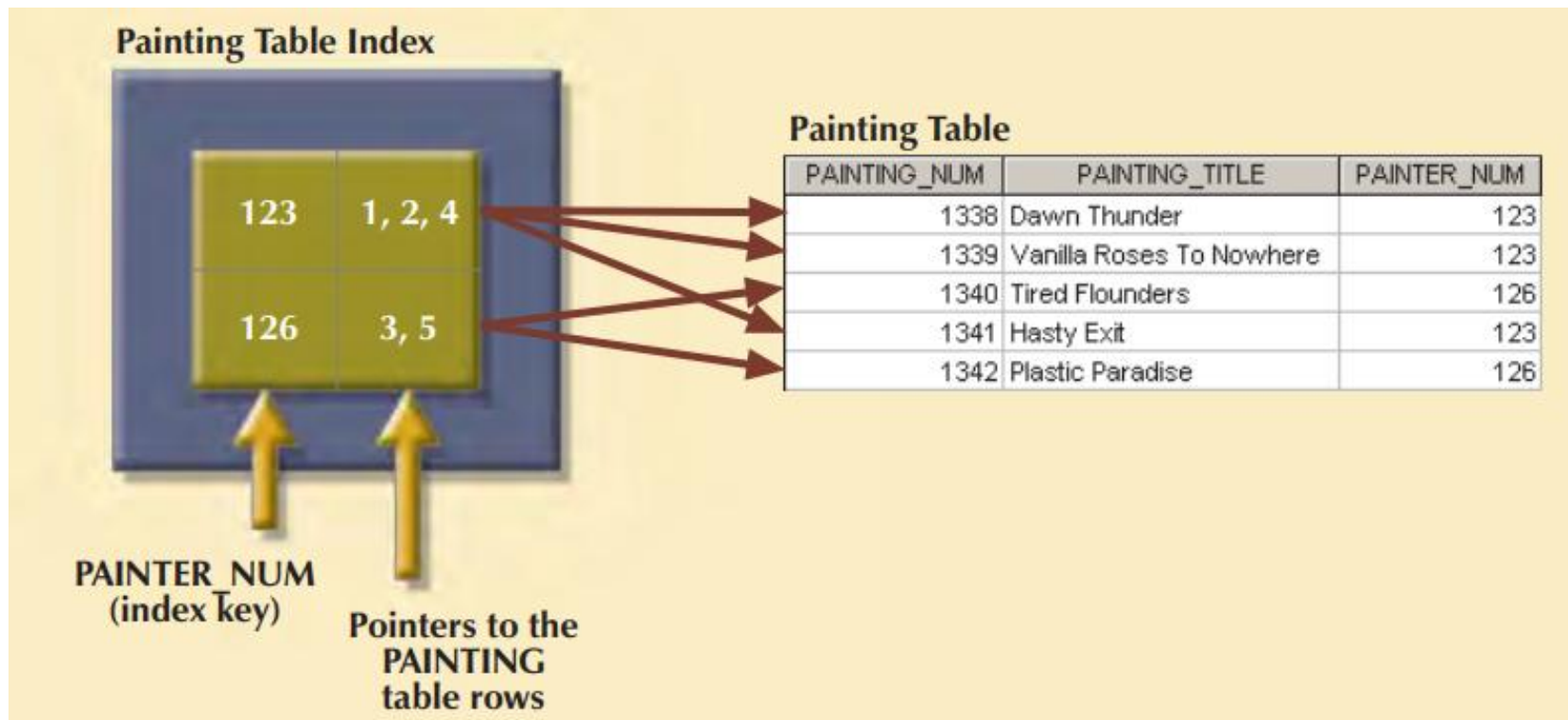
- Bạn muốn tìm 1 cuốn sách trong thư viện, bạn chọn lần lượt từng cuốn mà họ có cho đến khi tìm thấy hay bạn tìm theo danh mục (đã được lập chỉ mục theo chủ đề, tiêu đề và tác giả)?
- Bạn muốn tìm chủ đề “Data models” trong cuốn sách tham khảo môn học này, bạn lật tìm từng trang cho đến khi tìm thấy, hay bạn tìm cụm từ đó ở mục lục cuốn sách rồi mở đến trang cần tìm?



- **Một chỉ mục (Index) là một sự sắp xếp có trật tự được sử dụng để truy cập hợp lý các hàng trong một bảng.**
 - Sử dụng để nâng cao hiệu quả tìm kiếm
 - Tránh các giá trị cột trùng lặp.
- Một chỉ mục bao gồm một khóa chỉ mục và một tập hợp các điểm trỏ. Khóa chỉ mục là điểm tham chiếu của chỉ mục. Chính xác hơn, chỉ mục là sự sắp xếp có thứ tự của các khóa và điểm trỏ. Mỗi điểm trỏ, trỏ đến vị trí của dữ liệu được xác định bởi khóa.



■ Ví dụ:





■ Chú ý:

- Một bảng có thể có nhiều chỉ mục, nhưng mỗi chỉ mục được liên kết với chỉ một bảng.
- Khóa chỉ mục có thể có nhiều thuộc tính (chỉ mục tổng hợp).
- Indexes được tạo tự động khi các ràng buộc PRIMARY KEY và UNIQUE được định nghĩa trên các cột của bảng.



■ Tạo index:

CREATE [UNIQUE]INDEX indexname ON tablename(column1 [, column2])

- Unique: đảm bảo giá trị tại thuộc tính là duy nhất

■ Ví dụ:

Create index KT on Ket_qua(Kythi)

■ Xóa index: **DROP INDEX indexname ON tablename**

■ Ví dụ:

Drop index KT on Ket_qua;



Mô hình cơ sở dữ liệu quan hệ